

Manual del microblog Pulso

Instalación, uso, estructura de código, decisiones técnicas y relación con la PEC 5

STACK	React + Vite Node.js + Express MongoDB Atlas
ENTIDAD PRINCIPAL	Publicaciones con CRUD, imágenes y GIF
ACCESO	Administrador autenticado; visitantes sin registro
BASE DE DATOS	Driver oficial mongodb, sin Mongoose

Pulso es una mini aplicación full stack tipo X. Solo la cuenta administradora puede publicar, editar y borrar, además de decidir si cada publicación es pública o privada. El público puede leer, reaccionar y comentar en las publicaciones visibles sin registrarse.

1 Qué se ha construido

Una entrega pequeña pero completa, alineada con los contenidos reales del curso.

La entidad principal es publicaciones. El administrador realiza el CRUD completo desde React y los datos persisten en MongoDB. Las interacciones públicas se han diseñado como operaciones adicionales y no exigen almacenar cuentas de visitantes.

Funciones incluidas

- Listar publicaciones en cascada, de la más reciente a la más antigua.
- Autenticar al administrador mediante usuario, contraseña hasheada y JWT.
- Crear, editar y borrar publicaciones desde la interfaz.
- Elegir si cada publicación es pública o privada y cambiarlo al editarla.
- Añadir comentarios siempre públicos dentro de cada publicación visible.
- Eliminar comentarios desde la sesión de administrador.
- Subir, sustituir o retirar imágenes y GIF de hasta 15 MB.
- Alternar like o dislike por navegador sin crear perfiles públicos.
- Mostrar estados de carga, errores, confirmación de borrado y vista responsive.

Relación con los requisitos de PEC 5

Requisito	Cómo se cumple
App full stack	React/Vite consume una API Express conectada a MongoDB Atlas.
CRUD completo	POST, GET, PATCH y DELETE de publicaciones desde la interfaz.
IA documentada	README, PLAN, AGENTS, SKILLS, TASKS y reflexión crítica.
Pruebas	Node test, requests.http y colección Postman.
Variables	backend/.env.example y frontend/.env.example sin secretos.

2 Instalación paso a paso

La configuración local separa los secretos del código y utiliza dos procesos de desarrollo.

Requisitos previos

- Node.js 20 o posterior y npm.
- Un clúster de MongoDB Atlas activo y accesible desde el equipo.
- Una cuenta de base de datos con permisos sobre la base elegida.

Preparar las variables

1. Copia backend/.env.example como backend/.env.
2. Completa MONGODB_URI, MONGODB_DB y un JWT_SECRET largo y aleatorio.
3. Define ADMIN_USERNAME y ADMIN_PASSWORD solo para preparar el administrador.
4. Copia frontend/.env.example como frontend/.env.

```
PORT=4000
MONGODB_URI=mongodb+srv://...
MONGODB_DB=microblog_pec5
JWT_SECRET=frase-larga-y-aleatoria
CLIENT_ORIGIN=http://localhost:5173
```

Instalar y arrancar

5. Desde la raíz ejecuta npm run install:all.
6. Prepara el usuario con cd backend y npm run seed:admin. El script guarda un hash, nunca la contraseña en claro.
7. En la primera terminal ejecuta npm run dev:backend.
8. En otra terminal ejecuta npm run dev:frontend.
9. Abre http://localhost:5173.

El archivo .env real no debe subirse a GitHub. La entrega incluye únicamente ejemplos sin credenciales.

3 Uso de la aplicación

El muro es público; las operaciones de publicación aparecen únicamente después del login.

Como visitante

1. Abre la portada y recorre las publicaciones.
2. Pulsa el corazón para dar like o la flecha para dar dislike. Repetir la misma opción la desactiva; elegir la contraria cambia la reacción.
3. Despliega los comentarios, escribe un nombre de 2 a 40 caracteres y un texto de hasta 300 caracteres.

No se crea una cuenta. El navegador conserva un identificador aleatorio en localStorage únicamente para alternar reacciones. Borrar el almacenamiento del navegador genera una identidad local nueva; por eso no se presenta como sistema antifraude.

Como administrador

1. Pulsa Acceso de administración e introduce el usuario existente en MongoDB.
2. Tras el login aparece el formulario de publicación. El límite es 500 caracteres.
3. Al publicar puedes adjuntar una imagen o GIF. El archivo se limita a 15 MB y recibe un nombre aleatorio en el servidor.
4. Marca Publicación privada si solo quieres verla durante una sesión de administrador. Los comentarios siguen siendo públicos, pero quedan ocultos junto con el post.
5. Usa Editar para convertir una publicación pública en privada o al revés, además de cambiar texto, sustituir la imagen o retirarla.
6. Usa Eliminar junto a cualquier comentario para moderarlo.
7. Usa Borrar y confirma la acción. También se elimina del disco la imagen asociada.
8. Pulsa Salir para borrar el token del estado de React; el feed vuelve a mostrar solo publicaciones públicas.

El token se guarda solo en memoria. Recargar la página cierra la sesión, una elección sencilla y prudente para esta PEC.

Comprobación rápida

Acción	Resultado esperado
Visita sin login	Lee, reacciona y comenta; no ve crear/editar/borrar.
Login incorrecto	401 y mensaje de credenciales incorrectas.
Crear válido	201 y nueva publicación en la parte superior.
Post privado	Solo aparece con un JWT de administrador válido.
Editar/borrar	Solo con Authorization: Bearer TOKEN.

4 Arquitectura y recorrido de los datos

Frontend y backend son sistemas separados que se comunican con HTTP y JSON.

1 REACT	2 FETCH	3 EXPRESS	4 MONGODB
Evento y estado	Método, URL, JSON	Ruta, validación, controlador	Consulta con driver oficial

Ejemplo: crear una publicación

1. PostComposer mantiene un textarea controlado y llama a la función recibida por props.
2. services/api.js envía POST /api/posts con FormData y el token en Authorization.
3. Multer procesa el archivo image/* y genera un nombre seguro.
4. requireAdmin verifica el JWT y consulta que el usuario siga existiendo con rol admin.
5. El controlador valida el texto; el modelo ejecuta insertOne e incorpora adminId, autor, imageUrl y fechas.
6. La API responde 201 con la publicación creada. React crea un array nuevo y la añade al principio.

Ejemplo: reacción pública

El frontend envía tipo y visitorId. El backend retira la reacción contraria y añade la elegida, o retira la misma si ya estaba activa. Los arrays internos no se devuelven al navegador: la respuesta contiene solo los totales.

Esta separación evita que la vista conozca la URI de MongoDB y evita que el backend dependa de detalles del DOM.

5 Estructura del código

Cada carpeta tiene una responsabilidad concreta y localizable.

backend/	
scripts/seed-admin.js	crea o actualiza el único administrador
uploads/	imágenes locales ignoradas por Git
src/config/db.js	conexión compartida e índices
src/controllers/	valida y construye respuestas HTTP
src/middlewares/	JWT, 404 y errores
src/models/	operaciones MongoDB sin Mongoose
src/routes/	contrato de endpoints
src/utls/validation.js	reglas puras y reutilizables
test/validation.test.js	pruebas con node:test
frontend/	
src/components/	Header, Login, Composer, Post y Comments
src/services/api.js	URL, fetch, JSON, token y errores
src/App.jsx	estado general y coordinación
src/styles.css	sistema visual y responsive

Backend

db.js reutiliza una sola conexión y crea índices. Los archivos de models no son esquemas: contienen funciones como find, insertOne, updateOne y deleteOne. Los controladores desconocen los detalles del DOM y devuelven JSON y códigos HTTP.

Frontend

App.jsx conserva el array de publicaciones, la sesión y los estados de carga. Los componentes reciben datos y callbacks por props, como se trabajó en React. api.js centraliza las peticiones para que los componentes se concentren en la interfaz.

Comentarios en el código

Se han usado de forma selectiva: explican por qué se reutiliza la conexión, por qué el archivo model no implica Mongoose, por qué el middleware de error lleva cuatro parámetros y por qué la actualización de reacciones evita operadores incompatibles. No se comenta cada línea obvia, porque eso dificulta leer.

6 Modelo de datos y API

MongoDB conserva documentos; la API expone solo la representación necesaria para la interfaz.

Colección/campo	Tipo	Decisión
usuarios.usuario	String	Índice único y login sin registro público.
usuarios.password	String	Hash bcryptjs; nunca se recupera el texto original.
usuarios.rol	String	Debe ser admin para obtener y usar el token.
publicaciones.contenido	String	1 a 500 caracteres.
adminId	ObjectId	Procede del token, no del body.
imagenUrl	String/null	Ruta del archivo renombrado por Multer.
privado	Boolean	Oculto el post completo salvo al administrador.
reactions	Objeto	Dos arrays mutuamente excluyentes por visitorId.
comments	Array	Nombre, texto, autorAdmin, fecha e id; siempre públicos.

Endpoints principales

Método	Ruta	Acceso	Respuesta
GET	/api/posts	Público/admin	200 visibles
POST	/api/auth/login	Público	200 token / 401
POST	/api/posts	Admin	201 publicación
PATCH	/api/posts/:id	Admin	200 publicación
DELETE	/api/posts/:id	Admin	204 sin cuerpo
POST	/api/posts/:id/comments	Público/admin	201 publicación
DELETE	/api/posts/:id/comments/:commentId	Admin	200 publicación
POST	/api/posts/:id/reactions	Público/admin	200 publicación

Códigos relevantes

400 indica datos no válidos; 401 credenciales o token incorrectos; 404 endpoint, id o recurso no encontrado; 500 error interno. Un DELETE correcto usa 204 y no intenta enviar JSON, tal como se trabajó en el curso.

7 Decisiones técnicas justificadas

Las decisiones priorizan comprensión, coherencia con el curso y un alcance razonable para una semana.

Decisión	Por qué
Driver mongodb	Es el enfoque practicado y hace visibles ObjectId y operaciones CRUD.
JWT solo para admin	Protege las escrituras sin obligar a registrar visitantes.
Token en memoria	Reduce persistencia sensible; recargar cierra sesión.
Comentarios embebidos	Se leen siempre con el post y el volumen esperado es pequeño.
visitorId local	Permite alternar reacciones sin almacenar perfiles.
Una capa api.js	Evita repetir URL, cabeceras, serialización y tratamiento de errores.
Privacidad en API	MongoDB excluye posts privados; la ruta de imagen también valida el JWT.
Multer + FormData	Conserva GIF y otros image/* con límite y nombre aleatorio.
CSS propio	Mantiene la entrega comprensible sin añadir Tailwind innecesario.

Límites conocidos

- Un visitante puede borrar su localStorage y obtener otro identificador; no hay protección antifraude fuerte.
- Las imágenes se guardan en disco local; muchos proveedores no conservan ese disco tras un redesplicue.
- Embebidos en una publicación, comentarios y reacciones no son adecuados para una red social masiva.
- La contraseña inicial se introduce mediante variables locales; en producción conviene retirarlas después de preparar la cuenta.

Evolución posible

Si el proyecto creciera, movería comentarios y reacciones a colecciones independientes, añadiría paginación, rate limiting, moderación, cookies httpOnly para la sesión y pruebas de integración con una base temporal. Estas mejoras quedan fuera del alcance para no convertir la PEC en un sistema distinto del solicitado.

8 Uso crítico de IA

La IA aceleró trabajo repetitivo, pero el resultado se revisó por capas y se adaptó al aprendizaje del alumno.

Herramienta

Se utilizó ChatGPT/Codex para revisar el material del curso y el enunciado, proponer una arquitectura, generar una primera versión, crear pruebas y redactar documentación. La idea, las reglas de acceso y el alcance fueron definidos por el alumno.

Prompts principales

- Adaptar la PEC al driver oficial de MongoDB y eliminar Mongoose.
- Diseñar un microblog donde solo el administrador publique y el público interactúe sin registro.
- Separar rutas, controladores, acceso a datos, middlewares y componentes React.
- Revisar seguridad, ObjectId, operadores de arrays y estados de la interfaz.
- Comprobar build y pruebas y explicar decisiones y errores corregidos.

Errores y correcciones detectadas

- Una primera actualización de reacciones combinaba \$pull y \$addToSet sobre el mismo campo, lo que puede producir conflicto en MongoDB. Se dividió en ramas que modifican rutas compatibles.
- Se impidió que el body eligiera autor o adminId. El servidor obtiene ambos del usuario autenticado.
- Se separó el JWT del visitorId: el primero autoriza y el segundo solo controla una reacción local.
- La revisión visual redujo la cabecera editorial para que el muro aparezca antes en pantallas móviles.
- La privacidad se trasladó del comentario a la publicación completa. El backend filtra el post, bloquea sus interacciones y protege también su imagen; ocultarlo solo con CSS habría filtrado el contenido.

Qué aprendí

Revisar el flujo completo ayuda a detectar problemas que una sola capa oculta: una interfaz correcta no garantiza autorización, y una consulta válida no garantiza una experiencia coherente. La IA fue útil para acelerar, pero las pruebas y la lectura crítica determinaron qué código podía conservarse.

9 Pruebas y verificación

La comprobación combina funciones puras, build de producción y peticiones reales contra la API configurada.

Pruebas automáticas incluidas

```
npm test --prefix backend
npm run build --prefix frontend
```

El backend incluye cinco pruebas con node:test: publicación pública válida, privacidad multipart, publicación vacía, comentario público válido y reacción con identificador inválido. El frontend se construye con Vite; el build confirma que JSX, imports y estilos se procesan sin errores.

Pruebas manuales

1. Abre requests.http en VS Code con REST Client.
2. Ejecuta health, listado y login. REST Client reutiliza el token de la respuesta de login.
3. Crea una publicación, copia su id y prueba edición, reacción, comentario y borrado.
4. Créala privada, confirma que no aparece sin token, hazla pública y vuelve a convertirla en privada.
5. Prueba un id mal formado, texto vacío y comentario excesivo para confirmar 404/400.

También puede importarse microblog.postman_collection.json. El script de test del login guarda el token en la variable de la colección.

Resultados de esta entrega

Comprobación	Resultado
JSON y sintaxis backend	Correctos
Pruebas backend	5 superadas, 0 fallos
Integración Atlas	GIF, cambio público/privado, acceso y borrado verificados
Build frontend	35 módulos transformados, build correcto
Revisión visual móvil	Sin cortes; feed y comentarios visibles

10 Despliegue y entrega

El código está listo; las URLs finales requieren las cuentas del alumno en GitHub, Atlas y el proveedor de hosting.

Repositorio

- Inicializa Git en la raíz y comprueba que `.env` y `node_modules` están ignorados.
- Usa commits pequeños y descriptivos: estructura, API, autenticación, frontend, pruebas y documentación.
- Sube el repositorio sin `node_modules`, `dist` ni secretos.

Backend

- Crea un servicio Node en Render u otro proveedor compatible.
- Configura `MONGODB_URI`, `MONGODB_DB`, `JWT_SECRET` y `CLIENT_ORIGIN` en la plataforma.
- Usa `npm install` como build y `npm start` como inicio. No fijas `PORT` en producción.

Frontend

- Configura `VITE_API_URL` con la URL HTTPS de la API.
- Usa `npm run build` y publica la carpeta `dist`.
- Actualiza `CLIENT_ORIGIN` del backend con el dominio definitivo del frontend.

Persistencia de imágenes

- La carpeta `uploads` funciona localmente. Para producción usa Cloudinary, S3 u otro almacenamiento persistente y conserva la URL en MongoDB.

Lista final para Classroom

- Repositorio accesible y con historial de commits.
- URL del frontend y URL de la API funcionando.
- README, PLAN, AGENTS, SKILLS, TASKS, `.env.example` y pruebas.
- Este manual PDF y el código completo, sin credenciales reales.

Antes de entregar, ejecuta de nuevo todas las peticiones contra las URLs públicas. Un despliegue correcto no sustituye probar login, CRUD, comentarios y reacciones en producción.

11 Problemas frecuentes

Las causas más habituales se localizan comprobando configuración, red y orden de las capas.

Síntoma	Comprobación
La API no inicia	MONGODB_URI, acceso de red en Atlas y nombre de base.
Login siempre 401	Usuario, rol admin y hash generado por seed:admin.
React muestra Failed to fetch	API encendida, VITE_API_URL y CLIENT_ORIGIN.
Una ruta devuelve 404	Método, path, id de 24 hexadecimales y existencia del post.
Cambios no aparecen	Esperar respuesta correcta; revisar pestaña Network y consola de API.
El deploy del front llama a localhost	Cambiar VITE_API_URL y ejecutar un build nuevo.
Imagen desaparece tras deploy	El proveedor no conserva disco local; usar almacenamiento de objetos.

Método de diagnóstico recomendado

1. Prueba la consulta conocida desde requests.http.
2. Comprueba el endpoint y su estado HTTP.
3. Comprueba MongoDB y los recuentos matchedCount/deletedCount.
4. Solo después revisa el estado React y el DOM.

Este orden reproduce la forma de depuración practicada en las sesiones: aislar base de datos, API, petición y finalmente interfaz.

Conclusión

Pulso demuestra un flujo completo de datos, autorización y componentes sin ampliar innecesariamente el alcance. El proyecto puede explicarse por capas, probarse con herramientas del curso y evolucionar después del despliegue sin reescribir su base.